



Rapport de Projet

Software Engineering

Hugo CLAVEILLE

Master 1 Informatique

31 octobre 2025

Table des matières

1	Introduction	3
2	Présentation du problème et des objectifs	4
3	Conception et Implémentation	5
3.1	Structure générale du projet	5
3.2	Méthodes de compression	5
3.2.1	Méthode 1 : Entiers scindables	6
3.2.2	Méthode 2 : Entiers non scindables	6
3.3	Gestion des débordements	7
4	Résultats des Benchmarks	8
4.1	Méthodologie de Benchmark	8
4.2	Résultats Expérimentaux	8
4.2.1	Temps de compression et de décompression	9
4.2.2	Temps d'accès direct (get)	9
4.3	Resultats détaillés par distribution et nombre de bits	10
4.3.1	Nombre de bits	10
4.3.2	Distribution des données	12
4.4	Discussion des résultats	12
5	Temps de transmission	13
5.1	Estimation des temps de transmission	13

5.1.1	Transmission des données compressées	13
5.1.2	Transmission des données non compressées	13
5.2	Comparaison espace utilisé	14
5.3	Estimation des temps totaux de transmission	14
6	Problèmes observés et solutions proposées	16
6.1	Padding du dernier mot — zéros « fantômes » à la décompression	16
6.2	Nombre de bits pour l'index overflow	16

1. Introduction

Dans le cadre de l'Unité d'Enseignement de Software Engineering 2025, il nous a été demandé de réaliser un projet individuel visant à étudier et implémenter des méthodes de transmission optimisée de données. La transmission de tableaux d'entiers constitue un problème fondamental dans de nombreux systèmes informatiques et réseaux, en particulier pour les applications nécessitant un échange rapide et efficace de grandes quantités de données. Ce projet s'inscrit donc dans un contexte pratique où la performance et l'efficacité des transferts d'information sont essentielles.

L'objectif est d'étudier différentes méthodes de compression de ces tableaux afin de réduire le nombre de bits transmis tout en conservant un accès direct à chaque élément. La méthode choisie, appelée Bit Packing, représente chaque entier avec le minimum de bits nécessaires, regroupe ensuite dans des entiers standards pour la transmission. Deux variantes seront explorées : une autorisant l'écriture d'un entier compressé sur deux entiers consécutifs, et une qui ne le permet pas. Le projet inclut également des benchmarks pour évaluer les gains de performance.

La gestion des débordements (overflow) est un aspect crucial de ce projet. En effet, lorsque les entiers à compresser dépassent la capacité de bits allouée, il est nécessaire de mettre en place une stratégie efficace pour gérer ces cas. La méthode choisie consiste à utiliser des entiers supplémentaires pour stocker les valeurs qui ne peuvent pas être représentées dans le format compressé initial. Cette approche permet de maintenir l'intégrité des données tout en optimisant l'espace de stockage.

2. Présentation du problème et des objectifs

L'objectif principal du projet est de concevoir et comparer deux méthodes de compression de tableaux d'entiers en utilisant le principe du *Bit Packing*. Chaque entier est représenté à l'aide du nombre minimal de bits nécessaire, puis compacté dans un tableau d'entiers 32 bits standard.

Deux variantes ont été implémentées :

- **Méthode 1** : un entier compressé peut être réparti sur deux entiers consécutifs.
- **Méthode 2** : un entier compressé doit être contenu dans un seul entier.

Le but de ces deux approches est de déterminer laquelle offre le meilleur compromis entre temps de compression, temps de décompression et accessibilité directe à un élément donné. Des benchmarks automatisés ont été mis en place pour comparer ces deux méthodes sur des tailles de tableaux croissantes, distributions différentes ainsi que différents niveaux de compression (nombre de bits par entier).

3. Conception et Implémentation

3.1 Structure générale du projet

Le projet est structuré autour de trois composants principaux :

- Les classes `Method1Compressor` et `Method2Compressor` implémentant les deux méthodes de compression ;
- Un module de **benchmark** mesurant les performances pour différentes tailles de tableaux et valeurs de `maxbit` ;
- Un générateur de données permettant de produire des listes d'entiers selon plusieurs distributions (aléatoire uniforme, croissante, bruitée).

3.2 Méthodes de compression

Les deux classes `Method1Compressor` et `Method2Compressor` héritent d'une classe de base `CompressorBase`, qui définit l'interface commune pour la compression et la décompression. Chaque classe implémente les méthodes `compress()`, `decompress()` et `get()` selon la logique spécifique à chaque méthode.

Les deux méthodes utilisent des algorithmes différents pour gérer la répartition des bits et l'accès aux éléments compressés, que nous détaillerons ci-dessous. Nous traiterons plus tard de la gestion des débordements (overflow) lorsque les entiers à compresser dépassent la capacité allouée. Chaque entier compressé utilisant un nombre de bits inférieur ou égal à `maxbit` est annoté d'un préfixe 0 avant sa représentation binaire. Si l'entier dépasse cette capacité, il est annoté d'un préfixe 1 et stocké dans une structure dédiée aux débordements, décrite dans la section suivante. Dans les deux méthodes, la taille du tableau d'origine, le nombre de bits par entier (`maxbit`) et le nombre de bits utilisés pour chaque entier compressé en overflow sont stockés dans l'en-tête du tableau compressé pour faciliter la décompression.

3.2.1 Méthode 1 : Entiers scindables

Dans cette méthode, un entier compressé peut être réparti sur deux entiers consécutifs. Cela permet une utilisation plus efficace de l'espace lorsque les entiers à compresser ne s'alignent pas parfaitement avec les limites des entiers 32 bits. Par exemple, si chaque entier compressé utilise 10 bits, trois entiers compressés peuvent être stockés dans un entier 32 bits, et le quatrième sera scindé entre le premier et le deuxième entier suivant. (voir Figure 3.1) Cette approche maximise l'utilisation de chaque bit disponible, mais complique légèrement la logique d'accès aux éléments compressés. Pour l'entête, on stocke : Dans

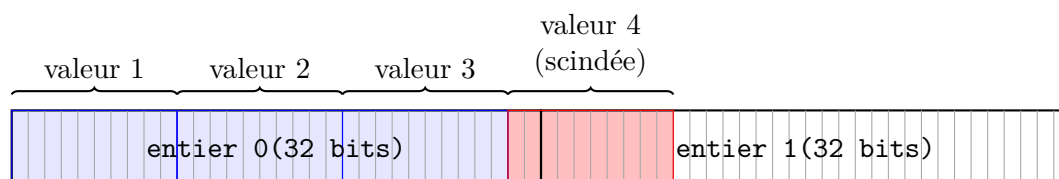


FIGURE 3.1 – Méthode 1 — un entier peut être scindé entre deux mots (ici : 2 + 8 bits).

un entier 32 bits : la taille du tableau original (n) Puis dans l'entier suivant : le nombre de bits par entier compressé (maxbit) sur 6 bits et le nombre de bits utilisés pour chaque entier compressé en overflow sur 6 bits.

Pour l'accès direct à un élément compressé, la méthode calcule la position exacte de l'entier dans le tableau compressé selon son indice et la taille des éléments, en tenant compte des éventuels débordements et du décalage de l'entête.

3.2.2 Méthode 2 : Entiers non scindables

Dans cette méthode, chaque entier compressé doit être entièrement contenu dans un seul entier 32 bits. Cela simplifie l'accès aux éléments compressés, car chaque entier peut être récupéré directement sans avoir à gérer la logique de scission entre deux entiers. Cependant, cette approche peut entraîner une utilisation moins efficace de l'espace, car certains bits dans les entiers 32 bits peuvent rester inutilisés si le nombre de bits par entier compressé ne s'aligne pas parfaitement avec 32.

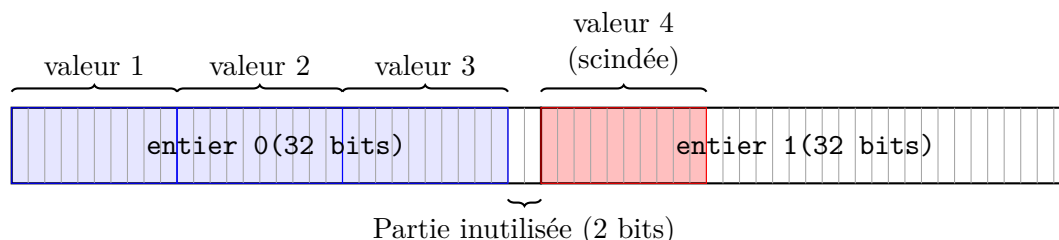


FIGURE 3.2 – Méthode 1 — un entier peut être scindé entre deux mots (ici : 2 + 8 bits).

Pour l'entete, on stocke : Dans un entier 32 bits : la taille du tableau original (n) Puis dans l'entier suivant : le nombre de bits par entier compressé (maxbit) sur 6 bits et le nombre de bits utilisés pour chaque entier compressé en overflow sur 6 bits.

Pour l'accès direct à un élément compressé, la méthode calcule la position exacte de l'entier dans lequel l'élément est stocké, en fonction de son indice et de la taille des elements, en tenant compte de l'entete. Puis la position de l'élément à l'intérieur de cet entier est déterminée par un simple calcul modulo.

3.3 Gestion des débordements

La gestion des débordements est gérée différemment selon la méthode choisie, mais utilisent le même principe de détection. Lorsqu'un entier à compresser dépasse la capacité maximale définie par `maxbit`, on préfixe la représentation binaire de son indice avec '1', ce qui permet de le différencier des entiers normaux préfixés par '0'. Puis cet entier est stocké dans une structure dédiée aux débordements. Lors de la décompression, si un entier avec le préfixe '1' est rencontré, la valeur correspondante est récupérée depuis cette structure à la fin de la décompression puis insérée à la bonne position dans le tableau décompressé.

Dans la Méthode 1, les entiers en overflow sont stockés à la suite des entiers compressés et peut dépasser sur plusieurs entiers 32 bits de la même façon que les entiers normaux.

Dans la Méthode 2, les entiers en overflow sont stockés dans des entiers 32 bits dédiés, chacun contenant un seul entier en overflow.

4. Résultats des Benchmarks

4.1 Méthodologie de Benchmark

Afin d'évaluer les performances des deux méthodes de compression développées, une campagne de mesures automatisées a été réalisée à l'aide d'un script Python dédié. Chaque configuration est définie par quatre paramètres principaux :

- le nombre de bits utilisés pour encoder chaque entier (`max_bits`);
- la taille du tableau d'entrée (`n`);
- la méthode de compression (`method1` ou `method2`).
- le type de distribution des entiers (`aléatoire uniforme`, `croissante` ou `bruitée`).

Pour chaque combinaison de paramètres, plusieurs itérations sont effectuées afin d'obtenir des moyennes représentatives des temps de traitement. Les trois opérations principales sont mesurées séparément :

- **Compression** : transformation du tableau d'entiers bruts en format compressé.
- **Décompression** : restitution du tableau original à partir du format compressé.
- **Accès direct** (`get`) : récupération aléatoire d'éléments sans décompression complète.

Chaque test est répété plusieurs fois pour éliminer les effets de cache ou d'aléa d'exécution. Les résultats sont sauvegardés dans un fichier CSV brut, puis agrégés en moyennes pour faciliter l'analyse et la génération de graphiques.

4.2 Résultats Expérimentaux

Après avoir exécuté les benchmarks selon la méthodologie décrite précédemment, nous pouvons maintenant nous intéresser aux résultats obtenus. Dans un premier temps, nous allons examiner les temps de compression et de décompression ainsi que les temps d'accès direct aux éléments compressés moyen pour chaque méthode sans tenir compte des différentes distributions de données ni du nombre de bits d'encodage.

4.2.1 Temps de compression et de décompression

Les temps de compression augmentent naturellement avec la taille du tableau et la complexité du codage (nombre de bits utilisés). Mais, malgré ce qu'on pourrait attendre, la **Méthode 1** montre des performances nettement supérieures à la **Méthode 2** pour des tailles de tableaux plus importantes. Cela s'explique par la flexibilité offerte par la possibilité de scinder les entiers entre deux mots, ce qui optimise l'utilisation de l'espace et réduit le nombre d'opérations nécessaires. Voir figure 4.1.

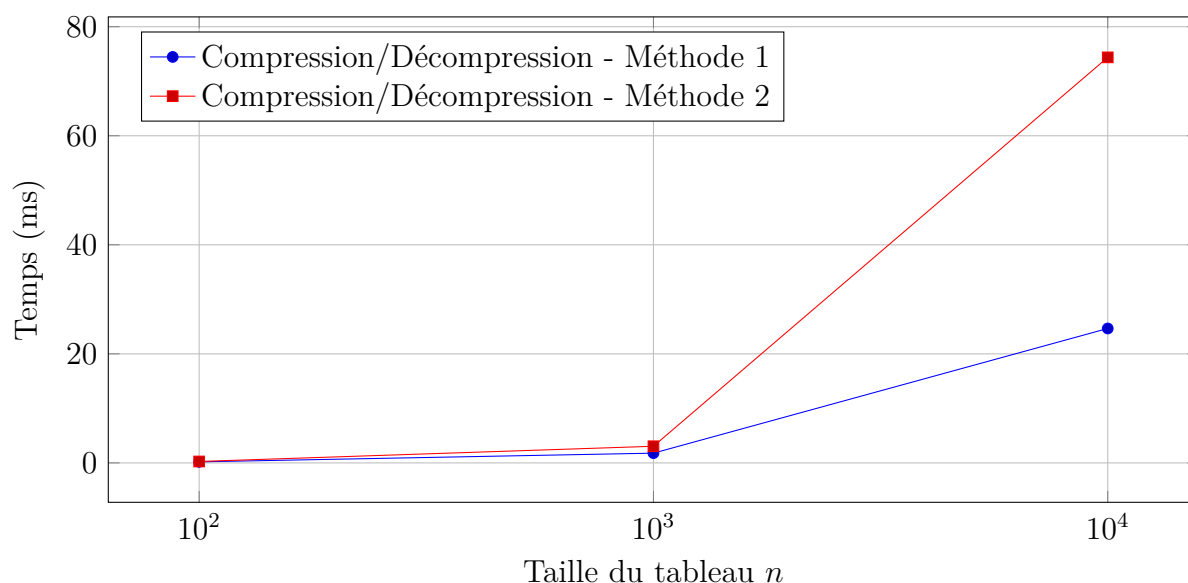


FIGURE 4.1 – Performance de la Méthode 1 : temps de compression et décompression moyen en fonction de la taille du tableau.

4.2.2 Temps d'accès direct (get)

La mesure des accès aléatoires est clairement étrange, le temps moyen d'accès direct via la méthode `get()` semble diminuer avec l'augmentation de la taille du tableau, ce qui est contre-intuitif. Cela pourrait être dû à la modification du nombre de bits utilisés pour l'encodage. Nous y reviendrons plus tard.

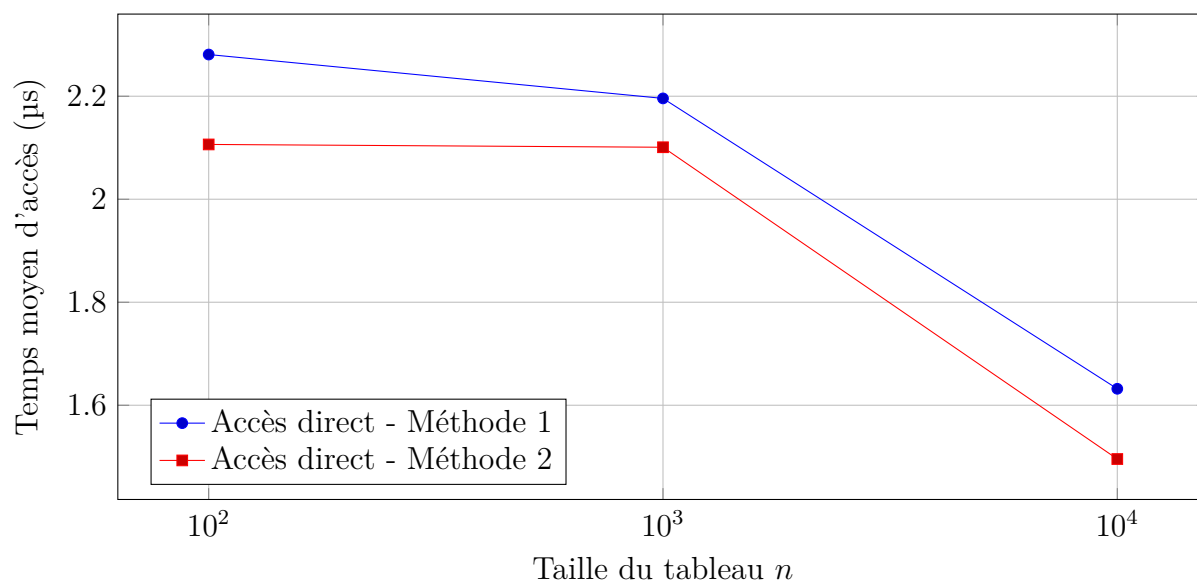


FIGURE 4.2 – Performance de la Méthode 1 : temps moyen d'accès direct en fonction de la taille du tableau.

4.3 Résultats détaillés par distribution et nombre de bits

Afin de mieux comprendre les performances des deux méthodes, nous avons analysé les résultats en fonction des différentes distributions de données et du nombre de bits utilisés pour l'encodage.

4.3.1 Nombre de bits

L'impact du nombre de bits sur les performances est significatif. En général, une augmentation du nombre de bits utilisés pour l'encodage conduit à une augmentation des temps de compression et de décompression. Les graphiques suivants illustrent ces variations en fonction de la taille du tableau et de la méthode utilisée.

On observe que la **Méthode 1** reste plus performante que la **Méthode 2** pour des tailles de tableaux plus importantes, indépendamment du nombre de bits utilisés. Cependant, pour des tailles plus petites, la différence de performance entre les deux méthodes est moins marquée.

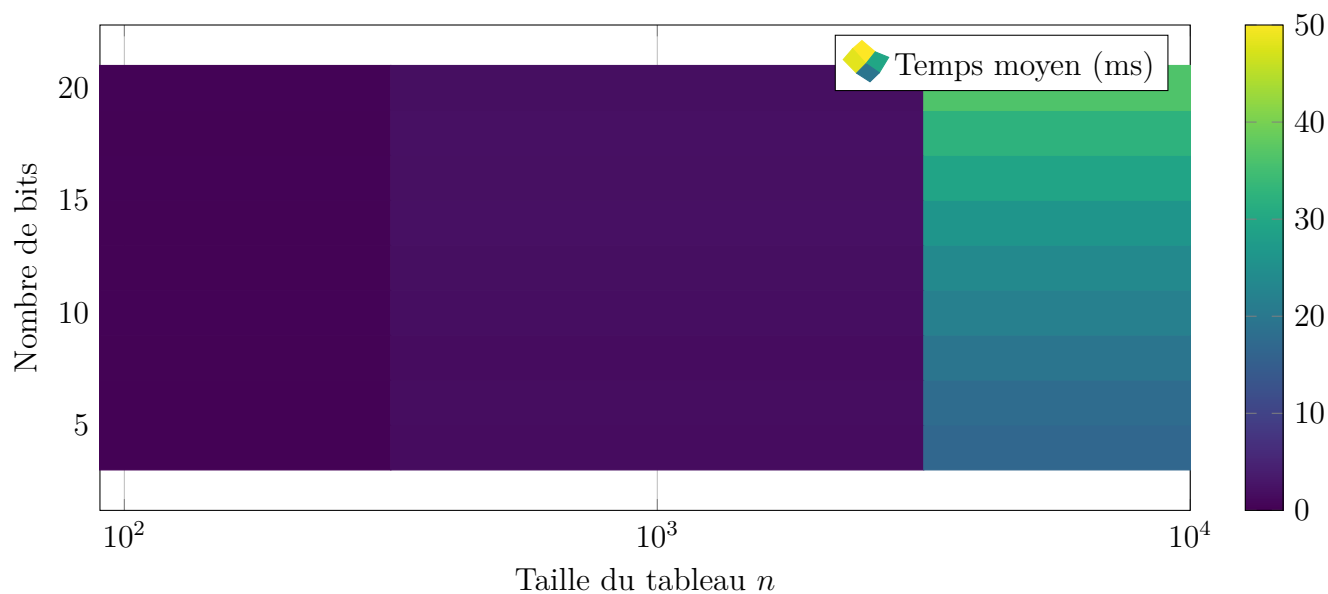


FIGURE 4.3 – Heatmap des performances en fonction de la taille du tableau et du nombre de bits utilisés sur la Méthode 1.

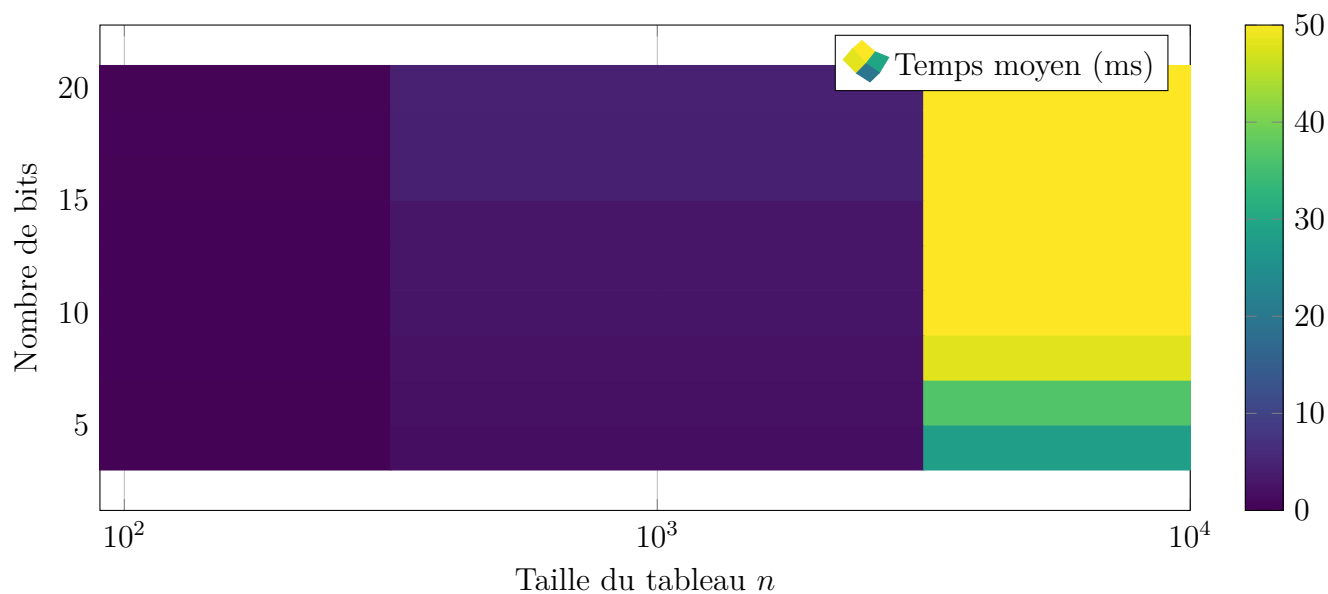


FIGURE 4.4 – Heatmap des performances en fonction de la taille du tableau et du nombre de bits utilisés sur la Méthode 2.

Maintenant que nous avons examiné les résultats en fonction du nombre de bits, nous allons analyser l'impact des différentes distributions de données sur les performances des deux méthodes.

4.3.2 Distribution des données

TABLE 4.1 – Comparaison des temps moyens (en ms) pour chaque méthode et distribution selon la taille du tableau n .

Distribution	n	Temps moyen (ms)	
		Méthode 1	Méthode 2
Uniforme	1000	0.19	0.27
	10000	1.72	3.04
	100000	24.75	74.61
Biaisée	1000	0.20	0.26
	10000	0.82	3.03
	100000	24.77	74.47
Avec outliers	1000	0.17	0.24
	10000	1.82	3.07
	100000	24.43	74.05

4.4 Discussion des résultats

Les résultats des benchmarks montrent clairement que la **Méthode 1** surpasse la **Méthode 2** en termes de temps de compression et de décompression, surtout pour des tailles de tableaux plus importantes. La flexibilité offerte par la possibilité de scinder les entiers entre deux mots permet une meilleure utilisation de l'espace et réduit le nombre d'opérations nécessaires. Cependant, la **Méthode 2** présente une logique plus simple pour l'accès direct aux éléments compressés, ce qui peut être avantageux dans certains contextes. L'analyse des résultats en fonction du nombre de bits utilisés pour l'encodage révèle que la performance diminue avec l'augmentation du nombre de bits, ce qui est attendu car cela augmente la complexité du codage. Enfin, l'impact des différentes distributions de données semble limité, suggérant que les deux méthodes sont robustes face à divers types de données.

5. Temps de transmission

Nous allons maintenant estimer les temps de transmission des données compressées en fonction des performances mesurées précédemment. En prenant la meilleure méthode (Méthode 1) et en utilisant les temps moyens de compression et de décompression obtenus lors des benchmarks, nous pouvons estimer les temps de transmission pour différentes tailles de tableaux et configurations de bits.

5.1 Estimation des temps de transmission

5.1.1 Transmission des données compressées

Pour estimer les temps de transmission, nous utilisons la formule suivante :

$$T_{total} = T_{compression} + T_{transmission} + T_{decompression} \quad (5.1)$$

où :

- $T_{compression}$ est le temps moyen de compression mesuré lors des benchmarks ;
- $T_{transmission}$ est le temps de transmission des données compressées, calculé en fonction de la taille des données et du débit du canal de transmission ;
- $T_{decompression}$ est le temps moyen de décompression mesuré lors des benchmarks.

Simplifiable en combinant les termes de compression et décompression en un seul terme $T_{processus}$:

$$T_{total} = T_{processus} + T_{transmission} \quad (5.2)$$

où $T_{processus} = T_{compression} + T_{decompression}$.

5.1.2 Transmission des données non compressées

Pour les données non compressées, le temps total de transmission est simplement le temps de transmission des données brutes :

$$T_{total_brut} = T_{transmission_brut} \quad (5.3)$$

Pour le temps de transmission brut, nous pouvons utiliser la formule suivante :

$$T_{\text{transmission_brut}} = \frac{T_{\text{brut}}}{D_{\text{canal}}} \quad (5.4)$$

où :

- T_{brut} est la taille des données brutes à transmettre ;
- D_{canal} est le débit du canal de transmission.

5.2 Comparaison espace utilisé

Pour comparer l'espace utilisé entre les données compressées et non compressées, nous calculons le ratio suivant :

$$R_{\text{espace}} = \frac{E_{\text{compressé}}}{E_{\text{brut}}} \quad (5.5)$$

où :

- $E_{\text{compressé}}$ est la taille des données compressées ;
- E_{brut} est la taille des données brutes.

Ce ratio nous permet de quantifier l'efficacité de la compression en termes d'espace utilisé.

La taille des données brutes pour un tableau de n entiers est donnée par :

$$E_{\text{brut}} = n \times 32 \text{ bits} \quad (5.6)$$

En prenant $n=10\,000$ éléments, on obtient :

$$E_{\text{brut}} = 10000 \times 32 = 320000 \text{ bits} \quad (5.7)$$

La moyenne de l'espace utilisé après compression calculé dans les benchmarks est, pour la Méthode 1, avec tout les encodages confondus, toutes les distributions confondues, et pour une taille de tableau de 10 000 éléments est de : 161059 bits

Ce qui nous donne un ratio d'espace utilisé de :

$$R_{\text{espace}} = \frac{161059}{320000} \approx 0.503 \quad (5.8)$$

Ce qui signifie que la compression réduit l'espace utilisé d'environ 49.7% par rapport aux données brutes.

5.3 Estimation des temps totaux de transmission

En prenant la variable t comme le temps de transmission par bit, nous allons a partir de quelle taille de chaine n la transmission compressée devient intéressante. Pour cela, il faut résoudre l'inéquation suivante :

$$T_{\text{compression/decompression}} + T_{\text{transmission}} < T_{\text{transmission_brut}} \quad (5.9)$$

En remplaçant les termes par leurs expressions respectives, on obtient :

$$T_{compression/decompression} + E_{compressé} \times t < E_{brut} \times t \quad (5.10)$$

Avec $E_{compressé}$, $latailledesdonnéescompressées$ et E_{brut} , $latailledesdonnéesbrutes$.

En remplaçant les tailles par leurs expressions en fonction de n , on a :

$$T_{compression/decompression} + (n \times 32 \times R_{espace}) \times t < (n \times 32) \times t \quad (5.11)$$

Avec R_{espace} le ratio d'espace utilisé calculé précédemment qui vaut environ 0.503.

En réarrangeant les termes et simplifiant, on obtient :

$$T_{compression/decompression} < (n \times 32 \times (1 - 0.503)) \times t \quad (5.12)$$

Enfin, on peut isoler n pour trouver la taille minimale du tableau à partir de laquelle la transmission compressée devient avantageuse :

$$n > \frac{T_{compression/decompression}}{32 \times (1 - 0.503) \times t} \quad (5.13)$$

En utilisant les valeurs moyennes de $T_{compression/decompression}$ obtenues lors des benchmarks pour une taille de tableau de 10 000 éléments, nous pouvons calculer la valeur de n pour différentes latences t .

TABLE 5.1 – Taille minimale du tableau n pour que la transmission compressée soit avantageuse en fonction de la latence t .

Latence par bit t (μ s)	Taille minimale du tableau n
0.01	1286
0.1	129
1	13
10	2

6. Problèmes observés et solutions proposées

Lors de l'implémentation et des tests, plusieurs problèmes pratiques sont apparus. Nous listons ci-dessous les symptômes, leur origine probable et la solution retenue ou proposée.

6.1 Padding du dernier mot — zéros « fantômes » à la décompression

Lorsque le flux compressé est segmenté en mots de 32 bits, le dernier mot est souvent partiellement rempli. Si la décompression lit bit-à-bit sans information sur la longueur originale, elle peut produire des zéros supplémentaires à la fin du tableau décompressé — éléments qui n'existaient pas dans l'original.

Absence d'information explicite sur le nombre d'éléments originaux ; le padding binaire (zéros) du dernier mot est interprété comme des éléments valides.

Stocker la **longueur originale** n dans l'en-tête du flux compressé (mot 0). Lors de la décompression, on lit d'abord n et on n'extrait que n entiers utiles, en ignorant tout padding. Cette solution est simple, robuste et peu coûteuse (32 bits).

6.2 Nombre de bits pour l'index overflow

Un problème qui m'a valu beaucoup de temps est le fait que si le nombre d'éléments en overflow dépasse la capacité d'indexation définie par $2^{\text{max_bits}}$, la décompression ne peut plus retrouver les valeurs en overflow. Pour résoudre ce problème, je fais en sorte que le i -ème entier en overflow dans le tableau compressé soit stocké dans le i -ème entier dans la zone overflow. Ainsi, l'indexation devient triviale et ne dépend plus du nombre d'éléments en overflow.